FYP VIVA DEFENSE SPECIAL GUIDE

PIZZA PLANET — DATABASE ARCHITECTURE

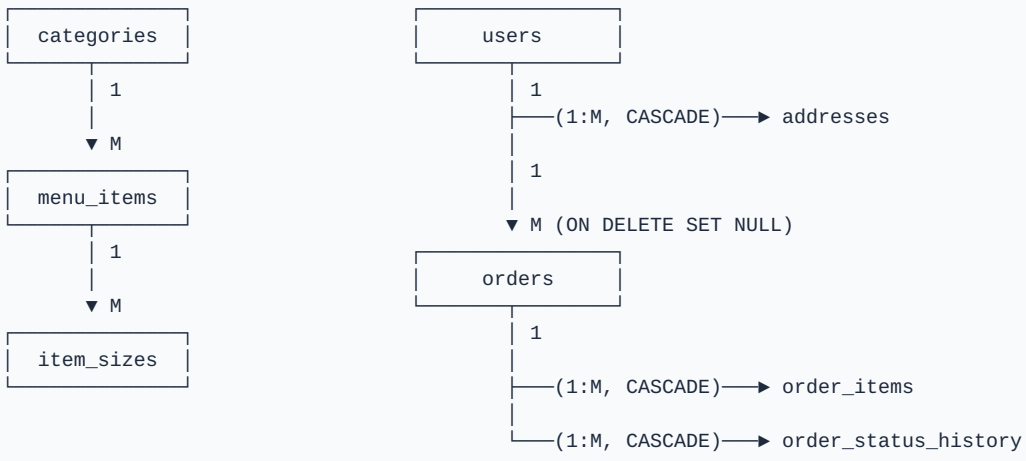
Relational Schema, ERD, Constraints, ACID Transactions & Top 15 Viva Questions

1. Database Overview & Core Metrics

The Pizza Planet database is engineered on **MySQL** / **MariaDB**, managed via the `mysql2/promise` driver with asynchronous connection pooling. It guarantees strict ACID compliance across financial and ordering transactions.

- **Database Name:** `pizza_planet`
- **Total Relational Tables:** 8 Tables (Fully Normalized to 3NF)
- **Primary Engine:** InnoDB (Supports Row-Level Locking, Foreign Keys & ACID Transactions)
- **Character Set & Collation:** UTF-8 (Supports multicharacter names, emojis, and international text)
- **Schema Source File:** `backend/database/schema.sql` (144 lines)

2. Entity-Relationship Diagram (ERD)



3. Comprehensive Table-by-Table Technical Breakdown

Table Name	Primary Key	Foreign Keys	Key Columns & Constraints	Business Purpose
users	`id` (INT AUTO_INCREMENT)	None	`email UNIQUE`, `phone UNIQUE`, `password VARCHAR(255)`, `role ENUM('customer','admin')`	Stores credentials and roles. Password stores 60-character Bcrypt hash (`$2b$10$...`).
addresses	`id` (INT AUTO_INCREMENT)	`user_id -> users(id)` **ON DELETE CASCADE**	`label VARCHAR(100)`, `address TEXT`, `is_default BOOLEAN`	Stores saved customer delivery locations. Deletes automatically if the user account is deleted.
categories	`id` (INT AUTO_INCREMENT)	None	`slug UNIQUE`, `label VARCHAR(100)`, `emoji VARCHAR(10)`, `display_order INT`	Menu categorization sections (Combos, Burgers, Pizza, Pasta, etc.).
menu_items	`id` (INT AUTO_INCREMENT)	`category_id -> categories(id)` **ON DELETE CASCADE**	`item_code UNIQUE`, `price DECIMAL(10,2)`, `has_sizes BOOLEAN`, `tags JSON`, `is_available BOOLEAN`	Master product catalog. Supports in-stock toggling and native JSON tags.
item_sizes	`id` (INT AUTO_INCREMENT)	`menu_item_id -> menu_items(id)` **ON DELETE CASCADE**	`size_key VARCHAR(20)`, `label VARCHAR(50)`, `price DECIMAL(10,2)`	Multi-size variations (Small, Medium, Large) with individual pricing for each size.
orders	`id` (INT AUTO_INCREMENT)	`user_id -> users(id)` **ON DELETE SET NULL**	`order_number UNIQUE`, `status ENUM(...)`, `total DECIMAL(10,2)`, `is_paid BOOLEAN`	Master financial order record. Preserves historical sales even if customer account is deleted.
order_items	`id` (INT AUTO_INCREMENT)	`order_id -> orders(id) CASCADE` `menu_item_id -> menu_items(id) SET NULL`	`item_name VARCHAR(200)`, `size_label VARCHAR(50)`, `unit_price DECIMAL(10,2)`, `quantity INT`	Immutable line-item snapshot of prices and quantities at time of purchase.
order_status_history	`id` (INT AUTO_INCREMENT)	`order_id -> orders(id)` **ON DELETE CASCADE**	`status VARCHAR(50)`, `notes TEXT`, `created_at TIMESTAMP`	Audit trail and chronological event log for live order tracking timelines.

4. Relational Constraints & Cascading Mechanics

> **EXAMINER TRAP: ON DELETE CASCADE vs ON DELETE SET NULL**
>
> Examiners frequently ask: *"Why didn't you put ON DELETE CASCADE on orders.user_id?"*
> **Answer:** If a customer deletes their user profile, their personal saved addresses are private data and should be deleted (`ON DELETE CASCADE`). However, business orders must **NEVER be deleted** because they are financial records required for accounting, tax auditing, revenue reports, and sales forecasting. By configuring `FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE SET NULL` , MySQL preserves the order records, financials, and line items forever while gracefully unlinking the deleted customer ID.

5. Database Indexes & Query Optimization

Without indexes, MySQL must execute a sequential **Full Table Scan (O(N))** across every record on disk. We defined strategic B-tree indexes:

```
CREATE INDEX idx_users_email ON users(email);
CREATE INDEX idx_users_phone ON users(phone);
CREATE INDEX idx_orders_status ON orders(status);
CREATE INDEX idx_orders_user ON orders(user_id);
CREATE INDEX idx_orders_phone ON orders(customer_phone);
CREATE INDEX idx_menu_items_category ON menu_items(category_id);
CREATE INDEX idx_order_items_order ON order_items(order_id);
```

- `idx_users_phone` & `idx_users_email`: Accelerates user login and duplicate customer verification from O(N) to O(log N).
- `idx_orders_status`: Accelerates administrative dashboard queries filtering active or pending orders.
- `idx_order_items_order`: Enables instant joining of cart items when loading customer order details.

6. ACID Transactions During Checkout (routes/orders.js)

Food ordering involves multi-table state changes that must execute under strict **ACID guarantees**:

```
const conn = await pool.getConnection(); // 1. Check out dedicated TCP socket from pool
try {
  await conn.beginTransaction();          // 2. Start ACID Transaction

  // A. Insert into 'orders'
  const [orderResult] = await conn.query('INSERT INTO orders (...) VALUES (...)', [...]);
  const orderId = orderResult.insertId;

  // B. Insert into 'order_items' (Loop)
  for (const it of items) {
    await conn.query('INSERT INTO order_items (...) VALUES (...)', [orderId, ...]);
  }

  // C. Insert into 'order_status_history'
  await conn.query('INSERT INTO order_status_history (...) VALUES (...)', [orderId, 'pending', 'Order
placed']);

  await conn.commit(); // 3. Atomic commit: All 3 table writes written to disk together
} catch (err) {
  await conn.rollback(); // 4. Rollback: Discards all partial writes if any failure occurs
  throw err;
} finally {
  conn.release(); // 5. Crucial: Socket returned to pool to prevent connection leaks
}
```

7. Top 15 Database Viva Defense Questions & Winning Answers

Q1: Why did you use DECIMAL(10,2) for prices instead of FLOAT or DOUBLE?

Why examiner asks: Tests numerical precision and financial software engineering standards.

> Don't say: "FLOAT is fine too." FLOAT has binary rounding errors!

FLOAT and DOUBLE are approximate floating-point representations (IEEE 754) that cause precision errors (e.g., 0.1 + 0.2 = 0.30000000000000004). In financial calculations, rounding errors compound into discrepancies. DECIMAL(10,2) is an exact fixed-point numeric type that stores numbers with exact scale, guaranteeing 100% accuracy in accounting and revenue totals.

Q2: Why did you choose MySQL (Relational) over MongoDB (NoSQL)?

Why examiner asks: Evaluates database paradigm selection criteria.

> Don't just say "MySQL is what we learned". Focus on ACID and Relational Integrity!

E-commerce food ordering requires strict relational integrity (categories linked to items, orders linked to users and items) and multi-statement ACID transactions. MySQL guarantees Atomicity and Consistency at the engine level through foreign key constraints and atomic transactions. NoSQL document stores like MongoDB are designed for unstructured data and lack strict relational constraints.

Q3: What is Connection Pooling and why is it used?

Why examiner asks: Tests database performance, socket lifecycle, and resource management.

Opening a new MySQL TCP socket on every HTTP request takes 20-50ms of network handshake latency. A connection pool (we configured 10 connections) pre-allocates reusable persistent connections. Queries checkout an idle socket, execute, and release it back to the pool instantly. It also caps maximum concurrent connections, protecting the MySQL server from memory exhaustion.

Q4: Why do you snapshot item_name and unit_price in order_items instead of just referencing menu_item_id?

Why examiner asks: Tests historical data immutability vs database normalization.

In a normalized database, you might only store menu_item_id. However, menu items change over time: prices increase and items get renamed or discontinued. If we only stored menu_item_id and joined on menu_items, past customer orders from 6 months ago would incorrectly display today's new prices! Storing item_name and unit_price in order_items creates an immutable snapshot of the purchase at that exact moment.

Q5: How do you prevent SQL Injection at the database query level?

Why examiner asks: Standard security compliance question.

We use prepared statements with parameterized placeholders ('?'). The SQL statement template and the data parameters are sent in two separate phases. The MySQL parser compiles the SQL query structure before binding user parameters. Thus, even if user input contains malicious syntax like "' OR 1=1; DROP TABLE orders; --", MySQL treats it strictly as a literal string value, preventing code injection.

Q6: What is the purpose of the ENUM data type in your users and orders tables?

Why examiner asks: Tests schema-level constraint validation.

ENUM restricts column values to an explicit allowed list (e.g., role ENUM('customer','admin') and status ENUM('pending','confirmed','preparing','ready','out_for_delivery','delivered','cancelled')). It enforces domain integrity directly at the database engine level, preventing illegal status strings, and is stored internally as a 1-byte integer, optimizing storage and query performance.

Q7: Why did you store tags as JSON in menu_items?

Why examiner asks: Tests modern hybrid relational-document database capabilities.

Tags (e.g., ["Spicy", "Chef Special", "Popular"]) are multi-value attributes. Creating a separate item_tags join table would add unnecessary JOIN overhead for simple descriptive labels. MySQL 5.7+ provides native JSON columns with binary storage and validation, giving us the flexibility of document stores while maintaining relational integrity.

8. Live MySQL Terminal Cheat Sheet

```
-- 1. Connect to MySQL CLI
mysql -u root -p

-- 2. Select database
USE pizza_planet;

-- 3. Show all tables
SHOW TABLES;

-- 4. Verify password hashing (shows $2b$10$... Bcrypt hash)
SELECT id, name, phone, password, role FROM users LIMIT 3;

-- 5. Inspect orders and COD payment status
SELECT id, order_number, subtotal, delivery_fee, total, status, is_paid, created_at FROM orders ORDER BY id
DESC LIMIT 5;

-- 6. Inspect order line items
SELECT order_id, item_name, size_label, unit_price, quantity, subtotal FROM order_items LIMIT 5;

-- 7. Inspect order tracking audit history
SELECT order_id, status, notes, created_at FROM order_status_history ORDER BY id DESC LIMIT 5;
```